Chromium Browser Artifact Parser

Programmer's Manual

Version: 2026-Mar-1 | Last Updated: March 2026

Author: Jacques Boucher (jjrboucher@gmail.com)

Repository: https://github.com/jjrboucher/Chromium-Browser-Artifact-Parser

Table of Contents

1. Architecture Overview
2. Project Structure
3. Core Components
4. Module Reference
5. Data Flow
6. SQLite Query Reference
7. Extending the Application
8. Chromium Database Schemas
9. Timestamp Handling
10. Contributing Guidelines

1. Architecture Overview

Design Philosophy

The Chromium Browser Artifact Parser follows a modular architecture that separates concerns into distinct components:

- GUI Layer — Tkinter-based interface handling user interaction
- Parser Modules — Individual modules for each artifact type
- Data Processing — Pandas-based data manipulation and transformation
- Output Generation — Excel file creation using openpyxl

Technology Stack

Component	Technology	Purpose
GUI Framework	Tkinter / ttk	Cross-platform user interface
Data Processing	Pandas	DataFrame operations and data manipulation
Database Access	sqlite3	SQLite database queries (immutable/read-only mode)
Excel Output	openpyxl	Excel file creation and manipulation
JSON Parsing	json (stdlib)	Preferences, Bookmarks, and manifest.json parsing

Threading Model

The application uses Python's threading module to prevent UI freezing during processing. The main thread handles UI updates while the worker thread processes artifacts.

2. Project Structure

```
Chromium-Browser-Artifact-Parser/
├── browser-artifact-parser-GUI.py   # Main application entry point
├── browser-artifact-parser-GUI.exe  # Windows executable (optional)
├── requirements.txt                 # Python dependencies
├── README.md                        # Basic documentation
│
├── Classes/
│   ├── Preferences.py               # Preferences file parser class
│   └── ChromeExtensions.py          # Extensions manifest.json parser
│
├── Functions/
│   └── write_to_excel.py            # Excel output utility
│
├── JSON/
│   ├── bookmarks.py                 # Bookmarks JSON parser
│   └── bookmarks_hashing.py         # Bookmark hash utilities
│
├── SQLite/
│   ├── history.py                   # History database queries + cluster queries
│   ├── downloads.py                 # Downloads queries
│   ├── cookies.py                   # Cookies queries
│   ├── favicons.py                  # Favicons queries
│   ├── logindata.py                 # Login Data queries
│   ├── searchterms.py               # Search terms queries
│   ├── shortcuts.py                 # Shortcuts queries
│   ├── topsites.py                  # Top Sites queries
│   ├── WebData.py                   # Web Data queries (autofill, etc.)
│   └── webasssist.py                # Edge-specific WebAssist queries
│
└── images/
    └── browser_chromium_icon.png    # Application icon
```

3. Core Components

3.1 Main Application Class

File: browser-artifact-parser-GUI.py

Class: ModernChromeParserGUI

The main class orchestrates the entire application. Key methods:

Method	Purpose
setup_styles()	Configure ttk widget styles
setup_artifact_selection()	Initialize artifact checkboxes (22 artifacts)
create_widgets()	Build the GUI layout
run_parser_threaded()	Start processing in background thread
run_parser()	Main processing logic
get_dataframes()	Execute SQL and return DataFrame
process_search_terms()	Special handling for search terms
process_bookmarks()	Parse JSON bookmarks
process_preferences()	Parse Preferences file
reorganize_workbook()	Reorder Excel worksheets

3.2 Artifact Configuration

Artifacts are defined in a configuration dictionary. Each artifact has an enabled flag (default checkbox state) and a query identifier for processing logic. The current set of 22 artifacts is:

```
self.artifacts_config = {
    'History': {'enabled': True, 'query': 'History'},
    'History Gaps': {'enabled': True, 'query': 'History Gaps'},
    'History Clusters': {'enabled': True, 'query': 'History Clusters'},
    'Downloads': {'enabled': True, 'query': 'Downloads'},
    'Downloads Gaps': {'enabled': True, 'query': 'Downloads Gaps'},
    'Autofill': {'enabled': True, 'query': 'Autofill'},
    'Addresses': {'enabled': True, 'query': 'Addresses'},
    'Keywords': {'enabled': True, 'query': 'Keywords'},
    'Credit Cards': {'enabled': True, 'query': 'Credit Cards'},
    'Bank Accounts': {'enabled': True, 'query': 'Bank Accounts'},
    'Loyalty Cards': {'enabled': True, 'query': 'Loyalty Cards'},
    'Login Data': {'enabled': True, 'query': 'Login Data'},
    'Login Data Gaps': {'enabled': True, 'query': 'Login Data Gaps'},
    'Shortcuts': {'enabled': True, 'query': 'Shortcuts'},
    'Top Sites': {'enabled': True, 'query': 'Top Sites'},
    'Cookies': {'enabled': True, 'query': 'Cookies'},
    'FavIcons': {'enabled': True, 'query': 'FavIcons'},
    'Search Terms': {'enabled': True, 'query': 'Search Terms'},
    'Bookmarks': {'enabled': True, 'query': 'Bookmarks'},
    'Preferences': {'enabled': True, 'query': 'Preferences'},
    'Extensions': {'enabled': True, 'query': 'Extensions'},
    'Web Assist (Edge)': {'enabled': False, 'query': 'Web Assist'}
}
```

3.3 Query Mappings

The run_parser() method maps artifacts to their database files and query functions. History Clusters maps to six separate query functions, each producing a distinct worksheet.

4. Module Reference

4.1 Classes/Preferences.py

Purpose: Parse the browser's Preferences JSON file to extract user and configuration data. Public methods are unchanged from the previous version.

4.2 Classes/ChromeExtensions.py (New)

Purpose: Parse Chrome extension manifest.json files from the profile's Extensions/ subfolder.

Class: ChromeExtensions

```
class ChromeExtensions:
    def __init__(self, profile_path: str):
        self.extensions_path = os.path.join(profile_path, 'Extensions')
        self._manifests: dict[str, dict] = {}
        self._manifest_paths: dict[str, str] = {}
        self._scan()
```

The _scan() method walks the Extensions/ directory. For each extension ID subfolder, it identifies the latest version subdirectory (sorted alphabetically) and reads its manifest.json. Public methods:

Method	Returns	Description
get_manifest_paths()	dict[str, str]	Maps each extension ID to its manifest.json path
get_name(extension_id)	str | None	Extension display name
get_version(extension_id)	str | None	Installed version number
get_description(extension_id)	str | None	Extension description
get_author(extension_id)	str | dict | None	Author or developer field (checks both keys)
get_homepage_url(extension_id)	str | None	Homepage URL from manifest
get_extension_count()	int	Total number of extensions found

4.3 Functions/write_to_excel.py

Utility function to write DataFrames to Excel files. If the output file already exists, the function appends a new worksheet using openpyxl append mode. If it doesn't exist, it creates a new file.

4.4 JSON/bookmarks.py

Parses Bookmarks and Bookmarks.bak JSON files. Output columns: Source, Type, Folder Path, ID, Name, Date added (raw and converted), Date last used (raw and converted), Date Modified (raw and converted, folders only), URL.

4.5 SQLite/history.py (Updated)

Now exports seven functions instead of two. The original chrome_history() and chrome_history_gaps() functions are unchanged. Six new cluster functions have been added:

Function	Worksheet Name	Key Tables Used
chrome_history_clusters_overview()	Clusters Overview	clusters, clusters_and_visits, cluster_keywords
chrome_history_clusters_contents()	Clusters Contents	clusters, clusters_and_visits, visits, urls
chrome_history_clusters_search_term()	Clusters Search Term	clusters, clusters_and_visits, visits, urls, content_annotations
chrome_history_clusters_timeline()	Clusters Timeline	clusters, clusters_and_visits, visits, urls
chrome_history_clusters_duplicate_visits()	Clusters Duplicate Visits	cluster_visit_duplicates, visits, urls
chrome_history_clusters_comprehensive_export()	Clusters Comprehensive Export	clusters, clusters_and_visits, visits, urls, content_annotations, context_annotations

4.6 Other SQLite Modules

Module	Functions	Database File
downloads.py	chrome_downloads(), chrome_downloads_gaps()	History
cookies.py	chrome_cookies()	Network/Cookies
favicons.py	chrome_favicons()	Favicons
logindata.py	chrome_login_data(), chrome_login_data_gaps()	Login Data
searchterms.py	chrome_keyword_historyquery()	History
shortcuts.py	chrome_shortcuts()	Shortcuts
topsites.py	chrome_topsites()	Top Sites
WebData.py	chrome_autofill(), chrome_keywords(), chrome_masked_credit_cards(), chrome_masked_bank_accounts(), chrome_addresses(), chrome_loyalty_cards()	Web Data
webasssist.py	edge_webassist()	WebAssistDatabase

5. Data Flow

Processing Pipeline

1. User selects profile path and output file.
2. User selects artifacts to process.
3. run_parser_threaded() launches run_parser() in a background thread.
4. For each selected artifact:
    - SQLite-based artifacts: get_dataframes() connects to the database in immutable read-only mode, executes the SQL query, and returns a DataFrame.
    - History Clusters: Iterates over all six cluster functions, calling get_dataframes() for each, producing six separate worksheets.
    - Extensions: ChromeExtensions class scans the Extensions/ folder and builds a DataFrame from manifest data.
    - Search Terms: Merges History and Web Data queries to produce the combined Search Terms DataFrame.
    - Bookmarks: JSON parser processes Bookmarks and Bookmarks.bak files.
    - Preferences: Preferences class parses the JSON file; output is written as formatted text rows.
5. Each DataFrame is written to the output Excel file via write_excel().
6. Summary DataFrame (record counts + script version) is appended.
7. reorganize_workbook() moves Summary and Preferences to the front.

Database Connection

All SQLite connections use immutable mode to guarantee read-only access and prevent accidental modification of the original data:

```
conn = sqlite3.connect(f'file:{db_file}?immutable=1', uri=True)
```

6. SQLite Query Reference

Timestamp Conversion Formula

Chromium uses WebKit timestamps (microseconds since January 1, 1601):

```
datetime(timestamp/1000000-11644473600,'unixepoch') AS 'Decoded timestamp (UTC)'
```

Breakdown: /1000000 converts microseconds to seconds; -11644473600 is the offset from the 1601 epoch to the Unix epoch (1970); 'unixepoch' tells SQLite to interpret as a Unix timestamp.

Transition Type Decoding

The transition field is a 32-bit integer with multiple flags. The rightmost byte encodes the core navigation type (0=link click, 1=typed URL, 2=suggestion, 7=form submit, 8=reload, etc.). Bits 24-32 encode qualifiers such as Forward/Back button, From Address Bar, redirect chain start/end, and JS or server-side redirects.

Gap Detection Pattern

Used in history_gaps, downloads_gaps, and logindata_gaps. The LAG window function identifies differences between consecutive ROWIDs. A final UNION checks whether the highest existing record number is lower than the sqlite_sequence value, indicating deleted records at the end of the table.

Cluster Query Design Principles

All six cluster query functions follow these forensic design principles:

- Raw values alongside decoded values — Every field that requires interpretation (timestamps, boolean flags, transition types, page_end_reason) is presented twice: once as the raw stored value and once decoded. This allows an independent party to verify every interpretation.
- LEFT JOINs for optional tables — content_annotations and context_annotations do not have rows for every visit; LEFT JOINs are used to avoid losing visit records that have no annotations.
- INNER JOIN for content_annotations in Clusters Search Term — Only visits with non-empty search_terms are relevant for that query, so an INNER JOIN is appropriate.
- cluster_keywords note — The cluster_keywords table is empty in current Chrome builds. The Clusters Search Term query derives search term data from content_annotations instead. A commented-out companion query against cluster_keywords is included in the source for future reference.

Addresses Query

Uses multiple self-joins on the address_type_tokens table with type codes (e.g., type 3 = first name, type 5 = last name) to reconstruct complete address records.

Bank Accounts Query

Joins masked_bank_accounts with masked_bank_accounts_metadata on instrument_id to include usage count and decoded last use date.

7. Extending the Application

Adding a New SQLite Artifact

Step 1: Create the query module in SQLite/:

```
def chrome_new_artifact():
    worksheet = 'New Artifact'
    sql_query = '''
        SELECT column1,
               datetime(timestamp/1000000-11644473600,'unixepoch') AS 'Decoded ts (UTC)'
        FROM new_table ORDER BY timestamp
    '''
    return sql_query, worksheet
```

Step 2: Import in main script:

```
from SQLite.new_artifact import chrome_new_artifact
```

Step 3: Add to artifact configuration:

```
'New Artifact': {'enabled': True, 'query': 'New Artifact'},
```

Step 4: Add to chromium_queries mapping:

```
'New Artifact': [f'{self.profile_path}/Database', chrome_new_artifact],
```

Adding Additional Cluster Worksheets

New cluster query functions follow the same pattern as the existing six. Add the function to SQLite/history.py, import it in the main script, and add it to the clusters list in the 'History Clusters' branch of run_parser():

```
clusters = [chrome_history_clusters_overview,
            chrome_history_clusters_contents,
            ...
            chrome_new_cluster_query]
```

Each function in the clusters list is called in sequence; all resulting DataFrames are written to separate worksheets.

Adding a New Preferences Field

In Classes/Preferences.py, add a method following the established pattern:

```
def new_preference(self):
    try:
        value = self.prefs.get('parent_key').get('child_key')
    except (KeyError, IndexError, AttributeError, TypeError):
        value = 'not found'
    return value
```

Then add the field to the __str__ method's formatted output.

Adding Browser-Specific Handling

For browser-specific artifacts (like Edge's WebAssist), add to the separate edge_queries mapping in run_parser():

```
edge_queries = {
    'Web Assist (Edge)': [f'{self.profile_path}/WebAssistDatabase', edge_webassist],
    'New Edge Feature': [f'{self.profile_path}/EdgeDatabase', edge_new_feature],
```

```
}
```

8. Chromium Database Schemas

History Database

Key tables: urls (URL records), visits (visit records linked to urls), visit_source (origin of visit), segments (URL path segments), keyword_search_terms (search queries), clusters (cluster metadata), clusters_and_visits (visit-to-cluster mapping), cluster_keywords (per-cluster keyword labels — empty in current builds), content_annotations (per-visit search terms, categories, entities, visibility scores), context_annotations (foreground duration, page end reason), cluster_visit_duplicates (duplicate visit pairs).

Web Data Database

Key tables: autofill, addresses, address_type_tokens, masked_credit_cards, server_card_metadata, masked_bank_accounts, masked_bank_accounts_metadata, loyalty_cards, loyalty_card_merchant_domain, keywords.

Login Data Database

Key tables: logins (saved credentials), stats (usage statistics in newer versions).

Favicons Database

Key tables: icon_mapping (URL to favicon mapping), favicons (favicon metadata), favicon_bitmaps (favicon image data — 3-table join required).

Other Databases

Network/Cookies: cookies table. Top Sites: top_sites table. Shortcuts: omni_box_shortcuts table. WebAssistDatabase (Edge only): Edge-specific navigation history.

9. Timestamp Handling

WebKit Timestamp Format

Chromium uses WebKit timestamps: unit is microseconds, epoch is January 1, 1601 00:00:00 UTC, stored as a 64-bit integer.

SQLite In-Query Conversion

```
datetime(timestamp/1000000-11644473600,'unixepoch')
```

Python Conversion

```
from datetime import datetime, timedelta
def convert_webkit_timestamp(webkit_timestamp):
    base_date = datetime(1601, 1, 1)
    return base_date + timedelta(seconds=webkit_timestamp / 1_000_000)
```

Handling Special Values

```
CASE timestamp
    WHEN 0 THEN 'Never'
    ELSE datetime(timestamp/1000000-11644473600,'unixepoch')
END
```

Unix Timestamps

Some fields (such as autofill dates) use Unix timestamps rather than WebKit timestamps:

```
datetime(date_created,'unixepoch') AS 'Decoded date_created (UTC)'
```

10. Contributing Guidelines

Code Style

- Follow PEP 8 Python style guidelines
- Use descriptive variable and function names
- Include docstrings for all functions and classes
- Comment complex SQL queries extensively

SQL Query Standards

- Include header comments with author, date, and tested versions
- Decode all timestamp fields with consistent column naming
- Handle NULL values appropriately
- Use CASE statements for enumerated values
- Present raw values alongside decoded values so an independent party can verify the decoding
- Order results by timestamp when applicable

Testing New Features

- Test against multiple browser versions
- Test with empty databases (especially cluster tables, which are empty in some builds)
- Test with large datasets
- Verify timestamp conversion accuracy
- Check Excel output formatting

Reporting Issues

Contact: jjrboucher@gmail.com

GitHub Issues: https://github.com/jjrboucher/Chromium-Browser-Artifact-Parser/issues

Include browser type and version, operating system, full error messages, and sample database files if possible (redacted).

Appendix A: Value Lookup Tables

Visit Source Values

Value	Meaning
0	Synced
1 (or NULL)	Local
2	Extension
3	Firefox Imported
4	IE Imported
5	Safari Imported

Download Danger Types

Value	Meaning
0	Not Dangerous
1	Dangerous File
2	Dangerous URL
3	Dangerous Content
4	Maybe Dangerous
5	Uncommon Content
6	User Validated
7	Dangerous Host
8	Potentially Unwanted
9-23	Various security states

Page End Reason Values (context_annotations)

Value	Meaning
0	END_OTHER
1	END_RELOAD
2	END_NAVIGATION
3	END_STOP
4	END_CLOSE
5	END_NEW_NAVIGATION

Favicon Types

Value	Meaning
0	Invalid
1	Favicon
2	Touch Icon
3	Touch Precomposed Icon
4	Web Manifest Icon

Appendix B: Chromium Source Code References

- Transition Types: https://cs.chromium.org/chromium/src/ui/base/page_transition_types.h
- History Types: https://cs.chromium.org/chromium/src/components/history/core/browser/history_types.h
- Download Danger Types: https://source.chromium.org/chromium/chromium/src/+/main:components/download/public/common/download_danger_type.h
- Visit Annotations (cluster tables): https://chromium.googlesource.com/chromium/src/+/refs/heads/main/components/history/core/browser/visit_annotations_database.h
- History test schema: https://cocalc.com/github/chromium/chromium/blob/main/components/test/data/history/history.57.sql